Content-Based Fingerprint Retrieval System for Criminal Identification Using Deep Feature Extraction

A Project Report Submitted

in Partial Fulfilment of the Requirements for the Degree of

[DEGREE AND PROGRAMME]

Submitted by

[STUDENT NAME(S)]
[REGISTRATION NUMBER(S)]

Under the guidance of

[GUIDE NAME AND DESIGNATION]

[DEPARTMENT]
[INSTITUTION]
[CITY, COUNTRY]

[MONTH YEAR]

UNDERTAKING

I/We, [STUDENT NAME(S)], declare that the work presented in this report, entitled Content-Based Fingerprint Retrieval System for Criminal Identification Using Deep Feature Extraction, is our work carried out under [GUIDE NAME]. Sources and contributions of others have been acknowledged. This declaration and the final submission status must be reviewed and signed by the student(s).

Date: [DATE]
Place: [PLACE]

Signature(s): [STUDENT SIGNATURE(S)]

CERTIFICATE

This is to certify that the project report entitled Content-Based Fingerprint Retrieval System for Criminal Identification Using Deep Feature Extraction has been submitted by [STUDENT NAME(S), REGISTRATION NUMBER(S)] under the guidance of [GUIDE NAME] to [DEPARTMENT, INSTITUTION]. The certificate is subject to verification and signature by the authorised faculty.

[GUIDE SIGNATURE] [HEAD OF DEPARTMENT SIGNATURE]
Date: [DATE]

Acknowledgment

I/We thank [GUIDE NAME], [DEPARTMENT], and [INSTITUTION] for their guidance and support. We acknowledge the authors of the SOCOFing dataset, the open-source software used in this work, and the researchers cited in this report. [Replace or extend this acknowledgement with the contributors and support actually received.]

Abstract

This project investigates content-based retrieval of fingerprints using fixed-length deep embeddings. A DenseNet121 backbone, selected from twelve trained checkpoints using a sampled equal-error-rate (EER) comparison, maps preprocessed 128 by 128 pixel fingerprint images to 512-dimensional, L2-normalised vectors. An ArcFace objective is used during training. Embeddings and image-level metadata are stored in a cosine-similarity Pinecone index; nearest-neighbour search returns ranked candidate fingers. The processed SOCOFing-derived splits contain 38,808 training, 8,283 validation, and 8,179 test images. On 1,000 genuine and 1,000 impostor test pairs, the selected checkpoint achieved a sample EER of 0.0%. In a separate 100-person, 1,000-query retrieval experiment spanning validation and test subjects, the original second, fourth, and sixth positions matched the finger class in 99.9%, 99.7%, and 99.4% of queries, respectively, after removing the self-match. These outcomes are specific to the recorded samples and synthetic alterations; they do not establish forensic accuracy on independent latent prints. The system provides candidate retrieval for further assessment, not proof of identity.

Contents

UNDERTAKING 2

CERTIFICATE 3

Acknowledgment 4

Abstract 5

Contents 6

List of Figures 8

List of Tables 9

List of Abbreviations 10

Chapter 1: Introduction 11

1.1 Background and motivation 11

1.2 Problem statement 11

1.3 Aim and objectives 11

1.4 Scope and contributions 12

1.5 Organisation of the report 12

Chapter 2: Literature Review 13

2.1 Friction-ridge comparison 13

2.2 Learned fingerprint representations 13

2.3 CNN backbones and angular-margin learning 13

2.4 Dataset and research gap 13

2.5 Existing approaches and limitations 13

Chapter 3: Preliminaries 15

3.1 Fingerprint biometrics 15

3.2 Image preprocessing and quality 15

3.3 Convolutional neural networks 15

3.4 Embedding space and similarity 15

3.5 Verification and retrieval metrics 15

Chapter 4: Proposed Model 16

4.1 Overall architecture 16

4.2 Data-flow and workflow 16

4.3 Preprocessing pipeline 17

4.4 Dense feature extraction 17

4.5 Vector database design 17
4.6 Retrieval, sequence and user interaction 18
4.7 Feasibility, requirements and deployment status 18
4.8 Use cases and database relationships 19
4.9 Activity and use-case views 19
Chapter 5: Experiments 20
5.1 Dataset and split organisation 20
5.2 Training and inference 20
5.3 Verification experiment 21
5.4 Retrieval experiment 21
5.5 Testing methodology and checks 21
5.6 Software and hardware requirements 22
5.7 Interface design and verification boundaries 22
Chapter 6: Results 23
6.1 Model comparison by EER 23
6.2 DenseNet retrieval results 23
6.3 Interpretation and comparison 24
6.4 Limitations and unmeasured quantities 24
6.5 Ethical, legal, privacy and security considerations 25
Chapter 7: Conclusion & Future Scope 26
7.1 Conclusion 26
7.2 Future scope 26
References 27
Appendix A: Reproducibility and submission checklist 28
Appendix B: Suggested further evaluation table 28

List of Figures

Figure 4.1: System architecture: implemented components and candidate review 16

Figure 4.2: Level 0 data flow: image to candidate list 16

Figure 4.3: Fingerprint preprocessing pipeline 17

Figure 4.4: DenseNet121 feature extraction and projection 17

Figure 4.5: Search sequence for a saved query 18

Figure 4.6: Logical dataset and vector relationships 19

Figure 4.7: Activity view: generation and saved-query evaluation 19

Figure 4.8: Level 1 data flow for a saved query 19

Figure 5.1: Subject-disjoint processed dataset organisation 20

Figure 5.2: Illustrative interface layout, not an application screenshot 22

Figure 6.1: Exact-position retrieval accuracy for DenseNet121 ArcFace 24

List of Tables

Table A: Abbreviations 10

Table 2.1: Comparison of relevant approaches 14

Table 4.1: Logical vector record 18

Table 5.1: Processed dataset split sizes 20

Table 5.2: Verification and test evidence 21

Table 6.1: EER comparison of best checkpoints (1,000 genuine and 1,000 impostor pairs) 23

Table 6.2: Exact-position DenseNet retrieval results for 1,000 queries 24

Table B.1: Unmeasured evaluation items - placeholders 28

List of Abbreviations

Table A: Abbreviations

Abbreviation	Meaning
AFIS	Automated Fingerprint Identification System
API	Application Programming Interface
ArcFace	Additive angular-margin training loss
CBIR	Content-based image retrieval
CLAHE	Contrast limited adaptive histogram equalisation
CNN	Convolutional neural network
DFD	Data-flow diagram
EER	Equal error rate
FAR / FRR	False acceptance / false rejection rate
mAP	Mean average precision
ROC / AUC	Receiver operating characteristic / area under curve
ROI	Region of interest

Chapter 1
Introduction

Fingerprint impressions contain friction-ridge arrangements that can support comparison of an unknown print with known exemplars. In forensic practice, the examiner assesses the clarity and correspondence of ridge detail, documents the comparison and follows appropriate verification procedures. Automated searches can narrow a candidate list, but a similarity score alone cannot establish source identity [1], [2]. This project examines a research prototype that represents a fingerprint by a learned numerical vector and retrieves nearby vectors.

1.1 Background and motivation

Fingerprint collections can be searched with minutiae-based representations, ridge-image representations, or learned features. Minutiae such as ridge endings and bifurcations are informative but can be difficult to detect reliably in noisy, partial or distorted impressions. Fixed-length representations offer a convenient interface to vector search and can be computed without an explicit minutiae graph [3]. Their usefulness depends on training, data quality, and the similarity between evaluation data and the intended deployment setting.

Content-based image retrieval (CBIR) compares measurable image content rather than textual file names. In this system the content descriptor is a CNN embedding: the same processing and model must be applied to both the query and enrolled images. The index returns vectors with high cosine similarity and the associated metadata identifies candidate fingers.

1.2 Problem statement

Given a fingerprint image and a gallery of enrolled fingerprint images, retrieve a ranked set of candidate fingers whose learned image representations are similar to the query. The research challenge is to preserve finger-specific evidence under image alteration and suppress similarity between distinct fingers. The operational challenge is to retain vectors persistently and map each search result back to the correct source image and finger class.

1.3 Aim and objectives

The aim is to develop and evaluate a content-based fingerprint candidate-retrieval pipeline. The specific objectives are to preprocess fingerprint images consistently; train and compare CNN feature extractors; generate and normalise 512-dimensional embeddings; store image-level vectors and metadata in Pinecone; retrieve similarity-ranked candidates; and measure verification and identification behaviour using documented experimental protocols.

1.4 Scope and contributions

The completed work includes subject-disjoint dataset preparation, multiple pretrained backbone options, metric-learning training, cached embedding generation, persistent vector uploads and scripted retrieval experiments. A Streamlit demonstration operates on saved query metadata and invokes the query module; arbitrary image upload and an operational police database are proposed extensions. The forensic context motivates the project but the prototype has not been validated for criminal casework.

1.5 Organisation of the report

Following the reference thesis, Chapter 2 reviews related work, Chapter 3 establishes the background, Chapter 4 describes the proposed and implemented model, Chapter 5 documents experiments and testing, Chapter 6 presents measured results, and Chapter 7 gives conclusions and future scope.

Chapter 2
Literature Review

Fingerprint identification research spans friction-ridge evidence, feature extraction, biometric evaluation, and efficient similarity search. This review uses the cited sources to motivate design choices rather than claiming that results from one benchmark transfer to another.

2.1 Friction-ridge comparison

The National Institute of Justice Fingerprint Sourcebook describes ridge characteristics, development and preservation of impressions, documentation and examination methodology [1]. NIST highlights human factors in latent-print comparison [2]. These sources emphasise the importance of image quality, examination context and documented verification; they do not justify identifying an individual solely from a nearest-neighbour score.

2.2 Learned fingerprint representations

Engelsma, Cao and Jain propose DeepPrint, a compact fixed-length fingerprint representation incorporating domain knowledge such as alignment and minutiae detection [3]. Their system differs from the present project: this prototype uses a generic DenseNet backbone and does not implement DeepPrint alignment, minutiae supervision or the published large-scale protocol. The shared idea is to learn descriptors that support efficient vector comparison.

2.3 CNN backbones and angular-margin learning

DenseNet connects each layer to preceding layers within a dense block, encouraging feature reuse [4]. ResNet uses residual connections [5], whereas VGG explores deeper networks built from small convolution filters [6]. The repository implements all three as interchangeable backbones. ArcFace introduces an additive angular margin in a classification objective originally developed for face recognition [7]. The project adapts that objective to finger classes; the learned embedding head, rather than the training classifier, is used at inference.

2.4 Dataset and research gap

The SOCOFing paper describes 6,000 original fingerprints from 600 subjects together with labelled synthetic alterations [8]. Such alterations support controlled evaluation but are not equivalent to independently acquired latent crime-scene impressions. The key gap for this project is therefore external validity: low sample EER on held-out SOCOFing subjects must be followed by independent capture, operational-quality and open-set studies before any practical forensic use.

2.5 Existing approaches and limitations

Table 2.1: Comparison of relevant approaches

Approach	Useful property	Limitation or distinction
Ridge and minutiae examination	Interpretable local evidence	Quality and examiner-dependent assessment [1], [2]
DeepPrint	Compact learned descriptor	Different architecture and evaluation protocol [3]
DenseNet + ArcFace (this project)	512-dimensional descriptor and vector search	Dataset-specific validation; no explicit minutiae model

The comparison illustrates a design trade-off: local ridge evidence can be examined and documented directly, whereas a learned descriptor provides a compact representation for retrieval. The present implementation prioritises a fixed-size representation and does not expose the individual ridge correspondences that a forensic examiner would ordinarily inspect. Further work could combine retrieval with a separately validated minutiae-based comparison.

The differences in training data, capture conditions, galleries and evaluation protocols also prevent treating published results as direct baselines. This report therefore compares the project checkpoints only on identical sampled pairs and records the full protocol alongside the results.

Chapter 3
Preliminaries

3.1 Fingerprint biometrics

A finger impression records a portion of the friction-ridge surface. Global patterns such as arches, loops and whorls provide broad structure; local ridge endings, bifurcations and their spatial relationships provide finer detail. Acquisition can produce rolled, plain or latent impressions, with differences in pressure, rotation, coverage and background. This project operates on labelled dataset images rather than live sensor capture; the pipeline should not be assumed to handle operational latent impressions without validation [1], [2].

3.2 Image preprocessing and quality

The implemented preprocessing reads grayscale images, trims configured borders, optionally applies median denoising, applies CLAHE, optionally derives a crop using inverted Otsu thresholding and morphological closing, pads to a square and resizes to 128 by 128. Values are scaled to [0,1], cached as NumPy arrays and later replicated to three channels and normalised using ImageNet channel statistics. Cropping and denoising are configurable; their use must be established from the actual configuration rather than assumed.

3.3 Convolutional neural networks

A convolutional layer applies learned spatial filters to its input and generates feature maps. Nonlinear activations, normalisation and pooling enable successive layers to encode texture and broader spatial structure. Transfer learning starts from weights learned on a larger image dataset, then fine-tunes for fingerprint discrimination. The repository implements a projection from the backbone feature vector to a fixed embedding dimension followed by L2 normalisation.

3.4 Embedding space and similarity

Let f(x) be the CNN representation of preprocessed image x and W the learned projection. The system computes z = Wf(x) + b and e = z / ||z||2. A cosine score s(e1,e2) = e1 dot e2 lies in [-1,1] for nonzero unit vectors. A larger score indicates greater geometric similarity under the learned model; it is not a probability of identity or an evidential likelihood ratio.

3.5 Verification and retrieval metrics

Verification accepts a pair when its score crosses a threshold. FAR is the proportion of impostor pairs accepted; FRR is the proportion of genuine pairs rejected. EER is the point at which the two rates are equal or interpolated to equality. ROC AUC summarises discrimination over thresholds. For retrieval, precision at k is relevant items among the first k divided by k; recall at k is retrieved relevant items divided by all relevant gallery items. F1 is the harmonic mean of precision and recall. Average precision aggregates precision at relevant ranks and mAP averages that quantity across queries. The reported rank-position accuracy is different from recall at k or mAP.

Chapter 4
Proposed Model

The architecture combines an offline enrolment workflow with query-time retrieval. The experimental evaluation uses stored test or validation vectors as queries. The interface path for a new arbitrary uploaded image is a proposed integration, not a demonstrated end-to-end feature.

4.1 Overall architecture

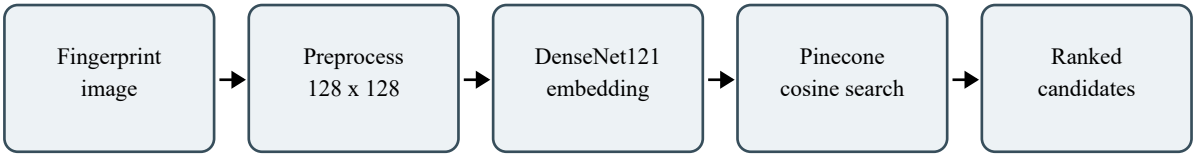


Figure 4.1: System architecture: implemented components and candidate review

The model checkpoint and preprocessing configuration determine the vector space. Enrolment and querying must use the same version. Pinecone namespaces separate model and dataset splits; local manifests map each namespace to its checkpoint and metadata. A reviewer interprets candidate results outside the automated ranking workflow.

4.2 Data-flow and workflow

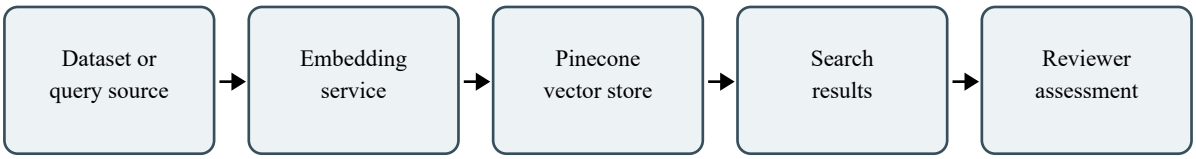


Figure 4.2: Level 0 data flow: image to candidate list

The external dataset supplies images and labels; the embedding process supplies vectors and limited metadata to the persistent vector store. A search request carries a vector to the same model-specific namespace and returns identifiers and similarity scores. Expert assessment is a contextual downstream activity, not an automated decision component.

4.3 Preprocessing pipeline



Figure 4.3: Fingerprint preprocessing pipeline

The optional ROI operation applies only when configured. Padding before resizing helps preserve aspect ratio. Image quality, missing ridges and severe geometric distortion can still affect the descriptor. Separate training augmentation produces variations during training; it is disabled when exporting evaluation embeddings.

4.4 Dense feature extraction

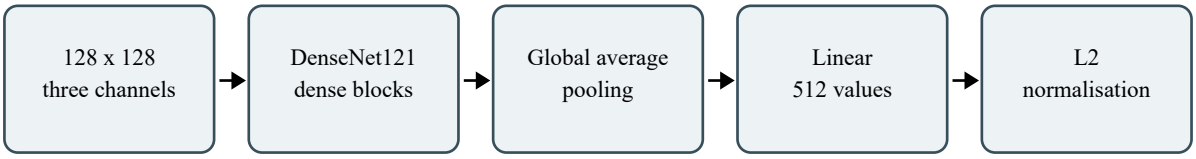


Figure 4.4: DenseNet121 feature extraction and projection

The selected checkpoint is checkpoints/densenet121_arcface/best.pt. DenseNet121 produces a 1,024-dimensional pooled feature; a learned linear head maps it to 512 dimensions and L2 normalisation yields the stored descriptor. ArcFace class weights are used during training, while inference returns the embedding. ResNet18 and VGG16 were retained as comparison backbones.

4.5 Vector database design

A Pinecone index is configured for 512-dimensional cosine vectors. Per-model and per-split namespaces reduce accidental mixing of incompatible embeddings. The uploader uses the split row as a stable vector ID and records class_id, split, model and available source-path and alteration metadata. The corresponding local metadata CSV records subject, hand and finger. These records are a logical mapping, not an implementation of a relational criminal-record database.

Table 4.1: Logical vector record

Field	Purpose
namespace	Model / split isolation
vector ID	Embedding row within the split
values	512 floating-point embedding components
class_id	Finger identity used in evaluation
split, model, source path	Provenance and reproducibility

4.6 Retrieval, sequence and user interaction

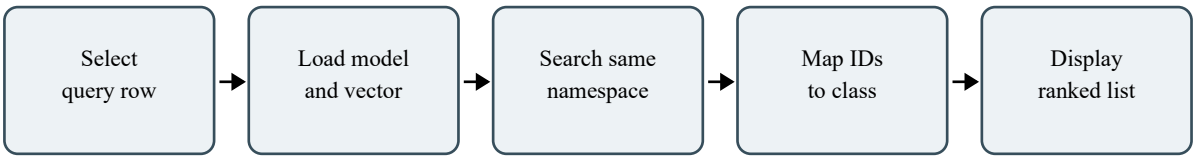


Figure 4.5: Search sequence for a saved query

The implemented command-line query loads a saved query row and the model, computes its embedding, asks Pinecone for top-k candidates and maps result identifiers to local gallery metadata. The separate person-evaluation script uses cached query embeddings and searches the image-level test or validation archive. The Streamlit demo selects an existing query number and top-k; an optional local Ollama explanation describes deterministic results and is not responsible for the match decision.

4.7 Feasibility, requirements and deployment status

Technical feasibility follows from the available Python, PyTorch, torchvision, OpenCV, NumPy, pandas, scikit-learn, Pinecone SDK and Streamlit components. A CUDA-capable NVIDIA GPU is useful for generating many embeddings but CPU inference is also supported. Pinecone requires account access, network connectivity, a compatible cosine index and a private API key. Storage and query costs depend on the service plan. Secure production enrolment, auditability, access controls and legal approval are required future engineering, not features demonstrated by the research scripts.

4.8 Use cases and database relationships

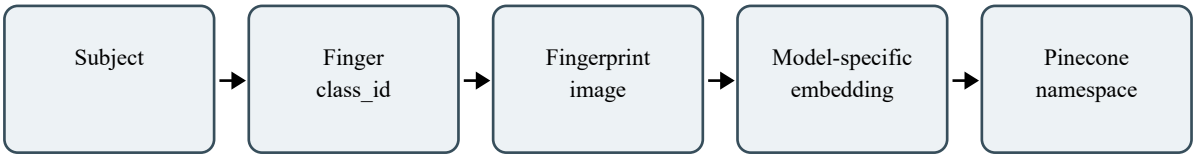


Figure 4.6: Logical dataset and vector relationships

The implemented roles are a researcher who prepares data and checkpoints, an operator who runs exports or experiments, and a viewer who selects a stored demo query. A future authorised examiner could submit an acquired print and review candidates. An image belongs to one finger class and one split; each image has one vector per checkpoint. A person has multiple finger classes. These logical relationships support data integrity but no separate personnel identity registry has been implemented.

4.9 Activity and use-case views

The implemented research workflow consists of preparing a dataset, generating embeddings, uploading the vectors, and searching a saved query. A proposed application use case would allow an authorised operator to supply a new image and inspect candidate fingerprints; that use case is not currently demonstrated as an upload feature.

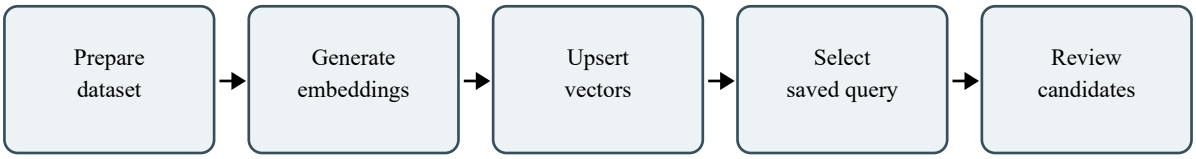


Figure 4.7: Activity view: generation and saved-query evaluation

The Level 1 flow in Figure 4.8 decomposes the saved-query process into selection, embedding, remote search, ID mapping and result display. The local metadata and the remote vectors must belong to the same checkpoint and split.

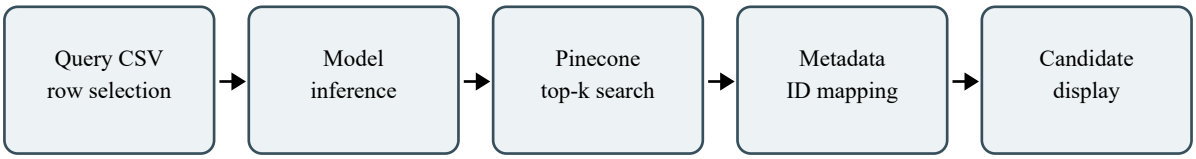


Figure 4.8: Level 1 data flow for a saved query

Chapter 5
Experiments

5.1 Dataset and split organisation

SOCOFing contains original fingerprints and synthetic alterations [8]. The project metadata uses subject_id, hand and finger to form class_id, so two different fingers of one person are different classes. The local processed data contain 55,270 images. A subject-disjoint split check found no overlap between the test subjects and either training or validation subjects. This reduces subject leakage between splits but does not establish independence between altered and original images of a single test finger.

Table 5.1: Processed dataset split sizes

Split	Images	Subjects	Finger classes per subject
Training	38,808	420	10
Validation	8,283	90	10
Test	8,179	90	10
Total	55,270	600	10

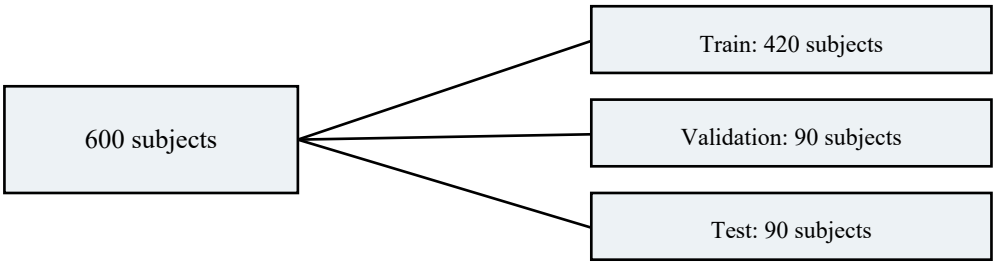


Figure 5.1: Subject-disjoint processed dataset organisation

Figure 5.1 shows the partition into three subject-disjoint splits. Real images and synthetic alterations can occur in a split. Files and counts reflect the local metadata at the time of the experiments; they should be regenerated if the dataset changes.

5.2 Training and inference

The training implementation supports progressive unfreezing: an initial phase trains the embedding head while the backbone is frozen, followed by fine-tuning at a lower backbone learning rate. ArcFace and triplet objectives are available; the selected checkpoint uses ArcFace. The project compares best saved checkpoints rather than training new models for this report. Training hyperparameters and epoch counts vary across runs; no single unverified value is asserted here.

Inference sets the model to evaluation mode, disables augmentation, computes embeddings using the trained head and validates normalisation. Local exports use memory-mapped arrays

and per-split manifests. The uploader uses stable IDs, batched upserts, progress tracking, remote vector counts and sample fetch checks. The observed execution environment used an NVIDIA GeForce RTX 3080 Laptop GPU with 8 GB memory; timing was not measured as a formal benchmark.

5.3 Verification experiment

The model-comparison script sampled 1,000 distinct genuine pairs and 1,000 distinct impostor pairs from the processed test CSV with random seed 42. Genuine pairs are different images sharing class_id; impostor pairs have different class_id. All twelve best checkpoints scored the same 2,000 pairs using cosine similarity. There were 3,179 unique participating images. EER is obtained by linear interpolation of the empirical ROC crossing; the separately reported nearest operating threshold is derived from the same test scores and should not be reused as a deployment threshold.

5.4 Retrieval experiment

For the DenseNet evaluation, 100 people were sampled reproducibly: 52 from test and 48 from validation. One random image was selected for each of ten fingers, producing 1,000 queries. Each query searched only its own complete split namespace with top_k=11. The exact query vector ID was removed. The first, third and fifth remaining candidates correspond to original positions 2, 4 and 6 when the self-match is ranked first. Every query in this run returned the self-vector first. The candidate class_id was compared to the query finger class_id; each person has ten trials per position.

This experiment searches the same collection from which each query was drawn. Removing self avoids the trivial identical-vector result, but other altered versions of the same source finger remain in the gallery. It measures closed-set finger retrieval under this sampled protocol; it is neither an independent-impression study nor an open-set false-identification assessment.

5.5 Testing methodology and checks

Table 5.2: Verification and test evidence

Test	Expected result	Recorded status
Pair generation	Distinct image IDs; label agrees with class_id	2,000 unique labelled pairs checked
EER implementation	Zero EER for separated synthetic scores	Unit test passed
Vector upload mapping	Stable row IDs and metadata preserved	Mocked upload and resume tests passed
Pinecone persistence	Remote count and sampled vectors agree	DenseNet test and validation verified
Query self exclusion	Remove matching vector ID	1,000 / 1,000 self ranked first
New arbitrary image UI	Acquire, preprocess and search uploaded image	Proposed; not established by saved-query demo

5.6 Software and hardware requirements

The implemented environment uses Python with PyTorch and torchvision for models [10], OpenCV and NumPy for image preparation, pandas for metadata, scikit-learn for evaluation, Pinecone for vector persistence [9], and Streamlit for the saved-query interface. A compatible GPU expedites inference. The API key is read from the environment or an ignored local .env file. The checkpoint, cached processed arrays and split CSV files are local dependencies; they are not included merely by cloning the source repository.

5.7 Interface design and verification boundaries

The existing demo presents retrieval settings, a selected known query, a top-k selector and an optional explanation checkbox. It requires generated index metadata and a threshold file. The displayed query identity is known for evaluation. No application screenshot was available for independent visual verification during report preparation; Figure 5.2 is an illustrative layout, not a screenshot of a working application.



Figure 5.2: Illustrative interface layout, not an application screenshot

Chapter 6
Results

6.1 Model comparison by EER

Table 6.1 reports the recorded same-pair comparison. Lower EER indicates a smaller error rate at the empirical equal-error operating point; AUC closer to one indicates better separation on these sampled scores. The 0.0% result is an observed sample value, not evidence that the true error rate is zero. DenseNet121 ArcFace was selected for subsequent embedding generation and retrieval experiments.

Table 6.1: EER comparison of best checkpoints (1,000 genuine and 1,000 impostor pairs)

Checkpoint	EER (%)	ROC AUC
densenet121 arcface	0.00	1.000000
densenet121 regularised	0.10	0.999982
densenet121 regularised margin20 (3)	0.10	0.999969
vgg16 regularised	0.10	0.999993
resnet18 regularised margin20 (6)	0.20	0.999958
vgg16 regularised long frozen	0.20	0.999967
resnet18 regularised (4)	0.20	0.999953
vgg16 regularised margin20	0.30	0.999972
densenet121 regularised long frozen (2)	0.50	0.999835
resnet18 regularised long frozen (5)	0.60	0.999789
resnet18 arcface	14.60	0.936460
vgg16 arcface v2	16.00	0.919528

The pairs are not necessarily statistically independent because one image or finger can appear in several pairs. Sampling was uniform over eligible finger classes for genuine pairs and over image pairs for impostors; this mixture influences the measured operating point. The test set includes synthetic alterations. Performance on independent captures, partial latent prints or a different population may differ substantially.

6.2 DenseNet retrieval results

Table 6.2: Exact-position DenseNet retrieval results for 1,000 queries

Original rank	Correct / 1,000	Accuracy	Mean cosine
2	999	99.9%	0.9425
4	997	99.7%	0.8969
6	994	99.4%	0.8637

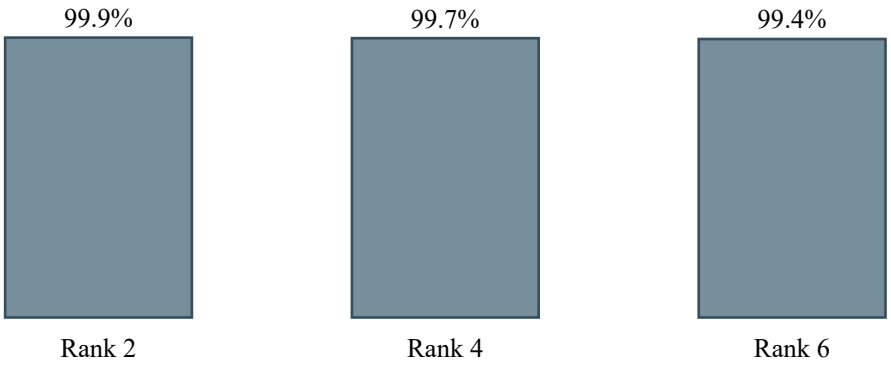


Figure 6.1: Exact-position retrieval accuracy for DenseNet121 ArcFace

The observed score is the fraction of the 1,000 queries for which the candidate at the specified position has the same class_id. Since each person contributed ten fingers, this also equals the mean of the per-person ten-query scores. It is not rank-k recall: a correct candidate at an earlier rank does not make a later position correct. Mean cosine is a similarity statistic, not a calibrated probability. The complete per-person and query-level results are retained as CSV files in outputs/person_evaluation/densenet121_arcface.

6.3 Interpretation and comparison

On the same retrieval protocol, the earlier ResNet18 ArcFace run yielded exact-position accuracies of 78.9%, 42.5% and 24.7% at positions 2, 4 and 6, respectively. DenseNet results are higher for this sampled benchmark. This observation supports the model choice for this project, but is not a head-to-head claim about operational false-positive rates. The benchmark includes many transformed versions per finger, and the impostor sampling does not represent every possible non-match in a large database.

6.4 Limitations and unmeasured quantities

No validated open-set false-identification rate, independent latent-print evaluation, end-to-end arbitrary-upload study, throughput or latency benchmark, storage-cost analysis, confidence interval, mAP, precision-recall curve or clinical/forensic deployment study is reported. The EER comparison selected a model using the test set, so that same test set is no longer an untouched final holdout for model selection. A new external subject-disjoint dataset is needed for an unbiased final assessment. A separate validation set should establish operational thresholds before the external test.

6.5 Ethical, legal, privacy and security considerations

Fingerprint images and embeddings are sensitive biometric information. A meaningful deployment must establish lawful authority, appropriate consent or other legal basis, retention periods, access restrictions, audit trails, encrypted transmission and storage, key management, incident response and deletion processes appropriate to the applicable jurisdiction. Embeddings can reveal identity-related information and should not be treated as anonymous merely because they are vectors. Source paths and finger identifiers in metadata also require protection.

Ranked candidates should be reviewed using an appropriate forensic workflow. A false match can affect a person even when the score is presented as a candidate; human review should be documented and protected against contextual bias [2]. Model performance should be investigated across image qualities and relevant population groups, using an appropriately governed independent evaluation. No legal compliance certification is claimed by this prototype.

Chapter 7
Conclusion & Future Scope

7.1 Conclusion

This work demonstrates a research pipeline for learning fixed-length fingerprint features, storing them in a persistent vector database and retrieving candidate matches. DenseNet121 ArcFace achieved the lowest sampled EER among twelve saved checkpoints and was selected for subsequent embedding generation. On the stated closed-set retrieval experiment, the selected exact positions matched finger class in 99.9%, 99.7% and 99.4% of queries. These findings are encouraging for the recorded experimental protocol but are insufficient to establish real-world forensic reliability.

7.2 Future scope

Priority next steps are to evaluate independent fingerprints collected under varied operational conditions; establish a new untouched final holdout after model selection; calibrate thresholds on separate validation data; and measure open-set identification, rank-k recall, mAP, latency and uncertainty. An upload interface should apply the same preprocessing and checkpoint version as enrolment, show traceable candidate metadata and enable an authorised expert to review results.

Further work may investigate calibrated quality gates, minutiae-aware reranking, explainable evidence presentation, domain adaptation and robustness to partial or low-quality prints. Governance work should address privacy, encryption, retention, access control, data provenance and audit logging before any sensitive deployment.

References

[1] National Institute of Justice, The Fingerprint Sourcebook, 2011. https://nij.ojp.gov/library/publications/fingerprint-sourcebook

[2] National Institute of Standards and Technology, Human Factors in Latent Print Analysis. https://www.nist.gov/programs-projects/human-factors-latent-print-analysis

[3] J. J. Engelsma, K. Cao and A. K. Jain, Learning a Fixed-Length Fingerprint Representation, arXiv:1909.09901, 2019. https://arxiv.org/abs/1909.09901

[4] G. Huang, Z. Liu, L. van der Maaten and K. Q. Weinberger, Densely Connected Convolutional Networks, Proceedings of CVPR, pp. 4700-4708, 2017. https://openaccess.thecvf.com/content_cvpr_2017/html/Huang_Densely_Connected_Convolutional_CVPR_2017_paper.html

[5] K. He, X. Zhang, S. Ren and J. Sun, Deep Residual Learning for Image Recognition, Proceedings of CVPR, 2016. https://arxiv.org/abs/1512.03385

[6] K. Simonyan and A. Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition, arXiv:1409.1556, 2014. https://arxiv.org/abs/1409.1556

[7] J. Deng, J. Guo, N. Xue and S. Zafeiriou, ArcFace: Additive Angular Margin Loss for Deep Face Recognition, Proceedings of CVPR, 2019. https://openaccess.thecvf.com/content_CVPR_2019/html/Deng_ArcFace_Additive_Angular_Margin_Loss_for_Deep_Face_Recognition_CVPR_2019_paper.html

[8] Y. I. Shehu, A. Ruiz-Garcia, V. Palade and A. James, Sokoto Coventry Fingerprint Dataset, arXiv:1807.10609, 2018. https://arxiv.org/abs/1807.10609

[9] Pinecone, Python SDK documentation, vector operations. https://sdk.pinecone.io/python/reference/sync-index.html

[10] PyTorch, Torchvision models documentation. https://pytorch.org/vision/stable/models.html

Appendix A: Reproducibility and submission checklist

The EER comparison is produced by python -m src.training.compare_eer. It writes comparison.csv, test_pairs.csv, per-checkpoint JSON reports and pair-score CSVs under outputs/eer_comparison. The retrieval study is produced by python -m src.retrieval.person_evaluation and writes person_scores.csv, query_results.csv and summary.json under outputs/person_evaluation/densenet121_arcface. These commands require the local dataset, checkpoints and, for retrieval, access to the Pinecone namespaces.

Before academic submission, replace the marked student names, registration numbers, degree, programme, institution, department, guide, date and signatures. Confirm dataset usage terms and institutional rules. Verify current cloud upload completion, namespace access and the experimental files independently. Add an actual screenshot only after visually inspecting the working application and label it as such. Update any formatting mandated by the institution.

Appendix B: Suggested further evaluation table

Table B.1: Unmeasured evaluation items - placeholders

Metric / study	Result	Evidence needed
mAP and recall at k	Not measured	Independent held-out query/gallery protocol
Open-set identification FAR / FRR	Not measured	Unenrolled identities and validation threshold
Latency and throughput	Not measured	Hardware, network, gallery size and repeats
Independent latent-print performance	Not measured	Appropriately governed external dataset